

Projeto Evolutivo

Requisitos das entregas AV1 e AV2

Front-end Frameworks — React • JavaScript • Vite

AV1

Protótipo local • 2,0 pts

AV2

API + Auth • 2,0 pts

Equipe

3 a 5 integrantes

Roteiro da apresentação

01 Panorama das duas entregas

02 Equipes, escopo e proporcionalidade

03 Tema do projeto

04 Stack tecnológica obrigatória

05 Checkpoints formativos

06 AV1 — protótipo com dados locais

07 AV2 — API e autenticação

08 APIs aprovadas e contingência

09 Repositório, README e commits

10 Apresentação e critérios de avaliação

11 Prazos, atrasos e integridade

12 Checklists finais

Regra central do projeto

A AV2 não é um projeto novo.

Ela conserva o tema, o repositório e a base funcional da AV1, mostrando com clareza o que foi aprimorado.

Tema

imutável entre AV1 e AV2

Repositório

um único, para as duas entregas

Equipe

mesma composição registrada

Evolução

documentada e demonstrável

Resumo das duas entregas

Mesmo produto, dois estágios de maturidade

ASPECTO	ENTREGA AV1	ENTREGA AV2
Produto	Protótipo front-end funcional	Evolução integrada e autenticada
Dados	Arquivo JS ou JSON local	API aprovada ou back-end da disciplina
Persistência	localStorage	API para dados remotos; localStorage quando fizer sentido
Navegação	React Router	React Router + área protegida
Autenticação	Não obrigatória	Obrigatória
Equipe	3 integrantes (ou 3–5 interdisciplinar)	Mesma equipe da AV1
Apresentação	Obrigatória	Obrigatória
Prazo / Valor	7 dias antes da prova • 2,0 pts	7 dias antes da prova • 2,0 pts

Formação e permanência das equipes

- Equipe padrão: exatamente 3 estudantes.
- Composição da AV1 é mantida na AV2.
- Trocas só com justificativa e autorização do professor.
- Cada integrante precisa de commits próprios.



Equipe interdisciplinar com Back-end: 3, 4 ou 5 integrantes — escolha informada na proposta e válida para as DUAS entregas.

Proporcionalidade do escopo

Equipes maiores entregam funcionalidades verticais adicionais



3 integrantes

Escopo-base

0 adicional



4 integrantes

Escopo-base

+1 funcionalidade vertical



5 integrantes

Escopo-base

+2 funcionalidades verticais

VALE COMO VERTICAL

Segundo perfil com área própria • painel de indicadores • nova entidade com fluxo completo • histórico/relatório com filtros • operação degradada e recuperação após falha da API

NÃO VALE

Mais cores, mais cartões ou páginas apenas informativas.

Uma funcionalidade vertical envolve interface, estado, regras e dados.

Tema do projeto

Livre, desde que aprovado na proposta e imutável entre as entregas

CRITÉRIOS DE APROVAÇÃO

- ✓ Adequado ao ambiente acadêmico
- ✓ Permite cumprir todos os requisitos
- ✓ Dados compatíveis com API aprovada ou back-end
- ✓ Aprovado na entrega da proposta
- ✓ Não muda entre AV1 e AV2

SUGESTÕES DE TEMA

- Catálogo, carrinho e lista de desejos
- Biblioteca ou organizador de estudos
- Catálogo de jogos ou criaturas
- Guia de filmes, séries e episódios
- Painel climático e atividades
- Dados brasileiros: CEP, feriados, bancos
- Agenda de eventos e inscrições
- Help desk / controle de filas
- Organizador de tarefas em equipe

Recomendado para equipes interdisciplinares: Sistema de Controle de Atendimento de um Laboratório de Análises Clínicas — recorte mínimo de Front-end detalhado no Apêndice A.

Tecnologia comum a todas as equipes

OBRIGATÓRIO

- ✓ React com componentes funcionais
- ✓ JavaScript
- ✓ Vite
- ✓ React Router
- ✓ CSS tradicional e/ou CSS Modules
- ✓ fetch nas requisições da AV2
- ✓ Git + repositório GitHub
- ✓ Um único repositório para as duas entregas
- ✓ package-lock.json versionado
- ✓ Instruções de execução no README

NÃO É EXIGIDO

- TypeScript
- Redux, Zustand ou outra lib de estado
- Axios
- Testes automatizados
- Deploy
- Next.js
- Avaliação da tecnologia interna do back-end

Nunca versionar: node_modules, senhas, tokens privados ou arquivos locais com segredos.

Entregas intermediárias sem nota

Formativas: não somam pontos, mas evitam surpresas na avaliação

4 sem. antes AV1

Proposta e equipe

Mapa de rotas, dados locais,
componentes iniciais e uma interação
funcionando

4 sem. antes AV2

Contrato de integração

Login, uma requisição de dados e
tratamento inicial de carregamento ou
erro

1

2

3

4

Tema, problema, público, integrantes,
opção de integração e 3
funcionalidades

2 sem. antes AV1

Protótipo parcial

API escolhida, endpoints, exemplos de
resposta, autenticação e
responsabilidades

2 sem. antes AV2

Demonstração técnica

O professor registra as orientações no Teams. A ausência em um checkpoint não reduz a nota automaticamente — a equipe segue responsável por todos os requisitos e prazos.

ENTREGA AV1 • 2,0 PONTOS

Protótipo funcional com dados locais

Componentização, estado, eventos, renderização, navegação e persistência local. Sem consumo de API nem dependência de back-end.

Requisitos funcionais mínimos da AV1

1

Página inicial que comunique a finalidade do produto

2

Ao menos 3 rotas úteis, uma delas de detalhe, cadastro, edição ou operação equivalente

3

Listagem de dados do domínio a partir de arquivo JS ou JSON local

4

Busca, filtro ou ordenação controlada por estado

5

Ao menos um formulário controlado com validação de campos obrigatórios

6

Duas ações que alterem dados ou estado (adicionar, editar, excluir, favoritar...)

7

Persistência no localStorage de um dado relevante do domínio

8

Mensagens condicionais: lista vazia, validação e resultado de ação

9

Navegação completa sem digitar endereços manualmente

React, organização e dados locais

REQUISITOS DE REACT E ORGANIZAÇÃO

- › Componentes funcionais
- › Props para comunicação
- › useState para estado mutável
- › Listas com chaves estáveis
- › Eventos tratados por funções
- › Componentes reutilizáveis
- › Separação páginas / componentes / dados / estilos
- › CSS legível e interface consistente

DADOS LOCAIS

```
src/  
  data/  
    dados.json
```

É permitido usar localStorage para gravar alterações. O README deve explicar quais dados são iniciais e quais são persistidos.

Não há número obrigatório de componentes: avalia-se a responsabilidade de cada parte, não a quantidade de arquivos.

Evidência de entrega da AV1

Enviar no Teams até o prazo



Repositório

Link do repositório do projeto



Commit final

Identificação do commit avaliado



Tag

Versão marcada como entrega-av1



README

Completo, conforme seção 10.2



Equipe

Nomes de todos os integrantes



Acesso

Público ou privado — se privado, liberar acesso ao professor antes do prazo

ENTREGA AV2 • 2,0 PONTOS

Evolução com API e autenticação

Dados remotos, autenticação, estados de requisição e organização mais próxima de um projeto profissional.

Requisitos comuns às duas opções

- | | | | |
|----|---|----|--|
| 01 | Preservar tema, repositório e funcionalidades essenciais da AV1 | 08 | Sessão preservada após atualizar a página, sem armazenar senha |
| 02 | Substituir ou complementar dados locais por dados de API | 09 | Logout implementado |
| 03 | Usar fetch, concentrado na pasta services | 10 | Tratar respostas 401 e 403 quando a API produzir esses estados |
| 04 | Estados visíveis: carregamento, sucesso, vazio e erro | 11 | URL-base da API centralizada em configuração |
| 05 | Ação de tentar novamente após falha recuperável | 12 | Incluir .env.example quando houver variáveis de ambiente |
| 06 | Autenticação conforme a seção 7.4 | 13 | Dados de contingência compatíveis com a interface |
| 07 | Pelo menos uma rota protegida | 14 | README atualizado: contrato, decisões, limitações e evolução |

Escolha exatamente uma opção de integração

Definida no checkpoint de contrato de integração

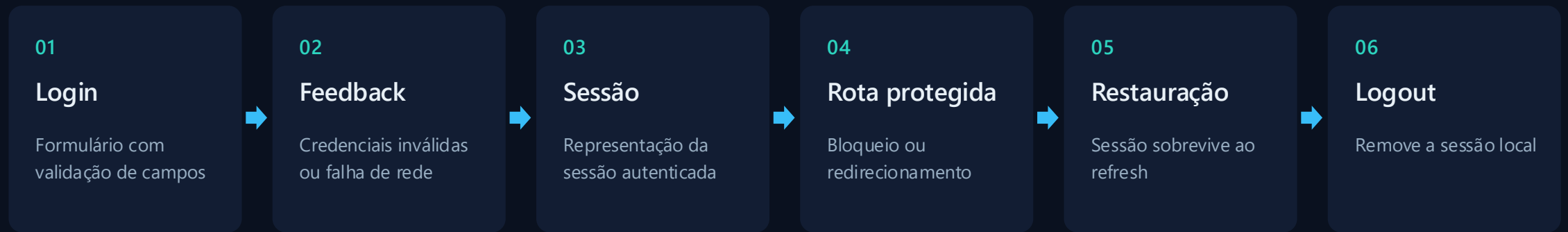
OPÇÃO A — API pronta aprovada

- Dados principais consumidos de uma API da lista aprovada
- Autenticação via endpoint de auth do DummyJSON
- Outra API no tema? DummyJSON pode servir só ao login/sessão
- Informar credenciais de demonstração na interface
- APIs somente leitura: favoritos e listas podem ser locais — deixar claro o que é remoto e o que é local
- Operações simuladas não devem ser apresentadas como persistência definitiva

OPÇÃO B — API do projeto de Back-end

- Servidor em qualquer tecnologia definida pelos professores/equipe
- Integração via HTTP, JSON e CORS no ambiente de desenvolvimento
- Back-end oferece autenticação + leitura, criação e alteração de estado
- Rejeitar com 401/403 o acesso indevido a operação protegida
- Documentar URL-base, endpoints, métodos, corpos, respostas e erros
- Back-end pode ficar em outro repositório, apontado no README

Autenticação obrigatória — fluxo mínimo



Autenticação não é esconder um botão.

Na Opção B o servidor também protege os endpoints correspondentes. Nada de senhas, tokens privados ou segredos no repositório.

OPCIONAL

Controle de dois ou mais perfis é opcional — exceto quando fizer parte do escopo adicional da equipe ou do contrato combinado com Back-end.

Evidência de evolução e de entrega

SEÇÃO OBRIGATÓRIA NO README: EVOLUÇÃO DA AV1 PARA A AV2

ASPECTO	SITUAÇÃO NA AV1	EVOLUÇÃO NA AV2	EVIDÊNCIA
Dados	JSON local	API escolhida	serviço/endpoint
Autenticação	não implementada	login e rota protegida	páginas e commits
Estados da interface	lista e vazio	carregamento, vazio e erro	componentes
Organização	estrutura inicial	páginas, serviços e estilos	pastas

EVIDÊNCIA DE ENTREGA DA AV2 NO TEAMS

Mesmo repositório da AV1 • commit final • tag entrega-av2 • README atualizado • contrato da API • .env.example (se aplicável) • link do repositório de Back-end na Opção B

Lista de APIs aprovadas

Verificada em 19 de agosto de 2026 — uso moderado e contingência local obrigatória

DummyJSON

Produtos, usuários, tarefas, receitas e posts

Tem autenticação; mutações simuladas, não persistidas

BrasilAPI

CEP, feriados, bancos, DDD, FIPE

Consultas; respeitar termos e evitar volume alto

PokéAPI v2

Criaturas, habilidades, itens e regiões

Somente leitura; manter cache ou fallback local

Open Library

Livros, autores, assuntos e capas

Uso educacional de baixo volume

TVmaze

Séries, programas, episódios e elenco

exige atribuição da fonte

Open-Meteo

Clima, previsão e planejamento

Consulta por coordenadas; observar licença

REGRAS DA LISTA

Outra API só após aprovação no checkpoint • chave secreta nunca no navegador ou repositório • serviços com cartão, plano pago ou cadastro pessoal não são aprovados • sem CORS, instáveis ou sem documentação podem ser recusados • o professor testará login e dois endpoints duas semanas antes da AV2.

Contingência para indisponibilidade da API

PARA TODAS AS EQUIPES

Preparar dados locais com a mesma estrutura usada pela interface.

A contingência serve para demonstrar a aplicação quando a rede ou o serviço externo falhar — ela não substitui a integração exigida.

ESPECÍFICO DA OPÇÃO B

- Demonstrar a integração no checkpoint intermediário
- Manter um exemplo de resposta de cada endpoint
- Fornecer instruções para iniciar o back-end
- Mensagem compreensível quando o servidor cair
- Nunca exibir apenas uma tela em branco

FALHA NO DIA DA APRESENTAÇÃO

Com motivo externo comprovado, o professor pode considerar o checkpoint, o código de integração e uma gravação curta feita antes da falha. Ainda assim, a equipe demonstra a interface com os dados de contingência.

Repositório, README e commits

ESTRUTURA RECOMENDADA

```
src/  
  components/  
  pages/  
  services/  
  data/  
  styles/  
  hooks/
```

A estrutura pode variar, desde que a equipe consiga justificá-la.

SEÇÕES OBRIGATÓRIAS DO README

- › Problema e público
- › Integrantes e contribuição
- › Funcionalidades
- › Rotas da aplicação
- › Tecnologias
- › Como executar
- › Dados e persistência
- › Contrato da API (AV2)
- › Evolução da AV1 para a AV2 (AV2)
- › Limitações e contingência
- › Uso de inteligência artificial

COMMITTS

- Cada integrante com commits próprios.
- Mensagens que descrevam a alteração
- Commits vazios, artificiais ou só no dia da entrega não comprovam contribuição
- Quantidade é analisada junto com conteúdo, regularidade e explicação oral

Apresentação obrigatória

Até 7 minutos por equipe, presencial, em cada entrega

DIVISÃO DO TEMPO



5 min — contexto e demonstração

2 min — perguntas do professor

Todos os integrantes falam e demonstram domínio da própria contribuição.

A apresentação usa a versão registrada no prazo; alterações posteriores são apenas complemento.

O PROFESSOR PODERÁ

Escolher quem explicará determinado trecho

Perguntar onde o estado é atualizado

Pedir explicação de uma requisição ou rota

Solicitar pequena alteração ao vivo

Perguntar sobre sugestão de IA registrada no README

Comparar a AV1 com a AV2

Critérios de avaliação de cada entrega

Total de 2,0 pontos por entrega

Funcionamento e qualidade do código

React, rotas, estado e persistência | evolução, integração, auth e organização



0,8

Interface e experiência de uso

Clareza, consistência e feedback | feedback de rede e autenticação



0,3

Documentação

Execução, equipe, dados e IA | contrato, evolução e contingência



0,2

Histórico e contribuição individual

Commits, coautorias e domínio da parte assumida



0,3

Apresentação e domínio

Demonstração da versão entregue, comparação e respostas



0,4

Equivalência: Opção A não perde pontos; Opção B não ganha pontos extras.

TOTAL 2,0

Prazos, atraso e integridade acadêmica

PRAZO E VERSÃO AVALIADA

Entrega exatamente 7 dias corridos antes da prova teórica, até 23h59 (salvo aviso no Teams).

A versão avaliada é o commit marcado como entrega-av1 ou entrega-av2.

PENALIDADE POR ATRASO

-0,1 ponto por dia corrido ou fração, limitado aos 2,0 pontos do projeto.

Repositório privado inacessível no prazo = entrega não acessível até a correção.

É PERMITIDO

- ✓ Consultar documentação e exemplos
- ✓ Discutir conceitos com colegas
- ✓ Usar IA para explicações, ideias, revisão e diagnóstico
- ✓ Aproveitar código gerado — revisando, testando, registrando e sabendo explicar

NÃO É PERMITIDO

- ✗ Copiar projeto, tutorial ou repositório inteiro
- ✗ Ocultar uso relevante de IA
- ✗ Entregar código que ninguém da equipe explica
- ✗ Inventar commits ou evidências
- ✗ Versionar credenciais, tokens ou dados pessoais

Checklist antes de entregar a AV1



Equipe e tema aprovados



Projeto usa React, JavaScript e Vite



Pelo menos três rotas úteis



Dados iniciais locais



Busca, filtro ou ordenação funciona



Formulário controlado e validado



Duas ações que alteram dados ou estado



Persistência relevante no localStorage



Mensagens condicionais na interface



Cada integrante com commits



README com equipe, execução, dados e IA



Tag entrega-av1 criada



Professor com acesso ao repositório

Checklist antes de entregar a AV2



Evolução da AV1 no mesmo repositório



Opção de integração aprovada



Dados obtidos por API



Carregamento, sucesso, vazio e erro visíveis



Existe ação de tentar novamente



Login, sessão, rota protegida e logout funcionam



Sem senhas nem segredos no repositório



URL-base centralizada



.env.example incluído quando necessário



Contingência local para a apresentação



README com contrato e tabela de evolução



Cada integrante com commits ou coautorias



Tag entrega-av2 criada



Todos preparados para apresentar e responder

Controle de Atendimento — recorte de Front-end

Opção recomendada para equipes interdisciplinares

Totem do cliente

Escolha entre SP, SG e SE, com emissão de senha local

Painel de chamadas

Senha atual, guichê e as cinco últimas senhas chamadas

Terminal do atendente

Ver fila, chamar, chamar novamente, iniciar e finalizar atendimento

MÁQUINA DE ESTADOS DO DOMÍNIO

EMITIDA

AGUARDANDO

CHAMADA

CHAMADA_NOVAMENTE

EM_ATENDIMENTO

ATENDIDA

NÃO_COMPARECEU

AV1: numeração no padrão 260819-SP001, sequência local por tipo, alternância simplificada entre SP e SE/SG explicada no README, não comparecimento após a 2ª chamada, persistência local e rotas distintas.

AV2: login de atendente, emissão de senha, consulta de fila/últimas chamadas, chamar próxima, iniciar e finalizar atendimento, tratando estados e erros do servidor.

Contrato lógico sugerido

Sem obrigar nomes de URL

OPERAÇÃO	MÉTODO	RESULTADO ESPERADO
Autenticar atendente	POST	sessão ou token e dados do usuário
Emitir senha	POST	senha criada com tipo, número e estado
Consultar últimas chamadas	GET	até cinco registros ordenados
Chamar próxima senha	POST	atendimento selecionado pelo servidor
Iniciar/finalizar atendimento	PATCH	novo estado e horários correspondentes
Consultar resumo	GET	indicadores, se fizerem parte do escopo

Prioridade, concorrência entre guichês e consistência são do back-end. O front-end exibe corretamente os resultados e as falhas.

Formulário da proposta

Documento inicial que autoriza o tema e a equipe

Nome provisório

API ou projeto de Back-end previsto

Problema e público

Três funcionalidades principais da AV1

Tema

Evolução planejada para a AV2

Integrantes (nome, disciplina de origem, papel inicial)

Funcionalidades adicionais para 4 ou 5 integrantes

Opção prevista para a AV2 — A ou B

Riscos e contingência

Entregar 4 semanas antes da AV1 — sem nota, mas condição para aprovação do tema e da equipe.

PRÓXIMOS PASSOS

Formar a equipe, escolher o tema e enviar a proposta.

1

Proposta e equipe aprovadas

2

Protótipo AV1 com dados locais

3

Contrato de integração

4

AV2 integrada e autenticada

Front-end Frameworks • Ciência da Computação • 2026.2 — dúvidas e orientações são registradas no Teams.